Research Intelligence Engine: Architecture and Empirical Characterization of a Desktop Multi-Source Scholarly Retrieval and LLM-Annotation Pipeline

Suryavansh Jaggi
IILM University, Greater Noida
suryavansh.jaggi.cs27@iilm.edu

Swati Vashisht
IILM University, Greater Noida
msswativashisht@gmail.com

***Abstract*—We characterize the Research Intelligence Engine (RIE), an open-source Electron/PostgreSQL desktop application that fans a query out to six scholarly-metadata providers, normalizes and deduplicates the results, stores them, and sends abstracts to a hosted LLM. Controlled experiments run RIE's unmodified modules against a deterministic emulator of the upstream APIs and a real PostgreSQL server. Search latency tracks the slowest adapter within 11 ms, but provider throttles impose a 350 ms floor and up to ≈2.6 s of delay on back-to-back searches, a hung provider blocks all results until its 15–30 s timeout, and arXiv failures are reported as empty successes. The deduplicator is order dependent (1 000 or 2 000 output records for identical inputs) and merges DOI-less records only when metadata strings match up to letter case. The AI client is serial, spaced by 2 s, retries only HTTP 429, and sends the literal text "null" for missing abstracts. A single user-recorded live Windows session (Semantic Scholar returned HTTP 429; eight papers analyzed and saved in 27 s) is consistent with these findings but is not a benchmark. No retrieval-quality or LLM-quality claims are made.**

***Index Terms*—scholarly metadata aggregation, federated search, record deduplication, LLM-assisted literature analysis, Electron, fault injection**

I. Introduction

Literature tools increasingly query open scholarly-metadata services directly: OpenAlex [1], Semantic Scholar [2], [3], Crossref [4], Europe PMC [5], PubMed via NCBI E-utilities [6], and arXiv [7]. Their coverage and metadata differ [8]–[10], so a multi-source client must reconcile schemas, tolerate partial failure and rate limits, and remove cross-provider duplicates. LLMs are also used to summarize papers, with a documented risk of unsupported output [11], [12].

The Research Intelligence Engine (RIE) packages these functions in one desktop workflow: concurrent retrieval from six providers, normalization, in-memory deduplication, PostgreSQL persistence with PDF acquisition, abstract-level LLM analysis, and a Markdown/LaTeX editor. Version 1.0.0 has no published evaluation, and its README describes a narrower, partly different system than the code (Section V). We ask: **RQ1** how the fan-out determines search latency and failure containment; **RQ2** how reliably the three-key deduplication resolves duplicates; **RQ3** what local components cost and how they scale; **RQ4** how the AI client schedules, retries and builds its prompt.

Evidence. The cloud sandbox used for the experiments blocked the scholarly APIs and the LLM endpoint, so the real modules were run unmodified against a deterministic upstream *emulator* with injected latency and faults, and a real PostgreSQL 16 server; these results are labelled **[S]**. One live session of the Windows desktop build, recorded by the author from in-app logs, is labelled **[L]** and reported separately (Table II). We did not measure real latency distributions, retrieval effectiveness, real overlap rates, or LLM output quality.

Contributions. (1) A code-grounded description and implementation audit of RIE. (2) A reproducible harness (fetch emulator, ground-truth corpus, fault injection, single-factor metadata perturbation) with a claim-by-claim audit and an independent re-run. (3) Findings on latency structure, failure reporting, request amplification and deduplication, notably that the deduplicator is non-confluent and its fallback key fails under every tested single-factor disagreement except letter case. (4) A harness-only relaxed-key resolver with a quantified precision/recall trade-off. (5) A labelled live session compared qualitatively with the emulated findings. The contributions are engineering and empirical; union-find resolution and fielded matching are standard record-linkage techniques [13]–[15].

II. Related Work

Open scholarly metadata. OpenAlex replaced Microsoft Academic Graph as an open index [1]; Semantic Scholar exposes papers, citations and full-text-derived fields [2], [3]; Crossref aggregates DOI metadata [4]; Europe PMC and PubMed cover the life sciences [5], [6]. Their coverage and completeness differ [8]–[10], and only part of the literature is open access (28 % or more [16]).

Federated search. Merging results of independent engines is federated search [17], often with rank fusion [18]; RIE does no fusion. Waiting for every backend exposes a fan-out to stragglers [19].

TABLE I
[C] PROVIDER ADAPTERS AS IMPLEMENTED (`INGESTION/SOURCES/*.JS`). "REQ." IS THE NUMBER OF UPSTREAM HTTP REQUESTS PER SEARCH FOR LIMIT k. TIMEOUTS ARE THE ADAPTER'S ABORTCONTROLLER VALUES.

Provider	Req.	Timeout	Behaviour noted in code
OpenAlex	1	15 s	$3k$ candidates, relevance sort; optional key
arXiv	1	30 s	$3k$ candidates; `ti:` OR `abs:`; $\geq$3 s between calls (module-wide)
Semantic Scholar	1(+1)	15 s	one retry after 2 s on HTTP 429; optional key
Crossref	1	15 s	scholarly-type filter; `mailto`
Europe PMC	$1{+}n$	15 s	Crossref look-up per hit with a DOI ($n \leq k$; 3 s timeout) for a PDF link
PubMed	2	25 s	ESearch then EFetch (XML), 350 ms spacing; optional key

Duplicate detection. Record linkage theory and techniques are surveyed in [13]–[15]; for systematic reviews, fielded procedures [20] and a dedicated tool finding 42.9 % more duplicates than a reference manager [21] have been reported.

LLM-assisted literature analysis. RAG [22], [23], PaperQA/PaperQA2 [24], [25] and scite [26] ground or classify scientific text; RIE sends only title, abstract and an instruction to an open-weight model (`openai/gpt-oss-120b` [27]) without grounding.

III. SYSTEM ARCHITECTURE

Process model. RIE follows Electron's multi-process design (Figure 1). Renderer windows (workbench and editor) run with `contextIsolation` on and `nodeIntegration` off and reach the main process only through `window.electronAPI` (namespaces `library`, `papers`, `ai`, `document`, `window`, `settings`) exposed by `preload.js`. IPC handlers delegate to `paper-service.js` (ingestion, database, files), `ai-service.js` and `document-service.js` (Pandoc). API keys are stored in a plaintext `settings.json` and mirrored into `process.env`.

Adapters and canonical record. Each provider has an *adapter* (HTTP and parsing only) and a *normalizer* (total function to a twelve-field record: `id`, `source`, `source_id`, `title`, `abstract`, `authors[]`, `year`, `venue`, `doi`, `url`, `pdf_url`, `citation_count`; missing fields → `null`), bound in a registry so a provider is added with one entry. Provider-specific handling (Table I) includes OpenAlex abstract reconstruction from an inverted index, JATS stripping for Crossref, a Crossref look-up for each Europe PMC hit with a DOI to find a PDF link, and deliberate omission of PubMed PDF links.

Search pipeline. `searchAndStore(q, sources, k, sortBy)` (Figure 2) dispatches all adapters with `Promise.allSettled`, normalizes, ranks and keeps the top k per provider, deduplicates the union, assigns `randomUUID`s *after* deduplication, re-ranks, and inserts into PostgreSQL. Ranking is by year then citations (or the reverse), missing values last, so provider relevance order is discarded. The result carries `providerStats` and `providerFailures`.

Persistence, PDFs and AI. One table `papers` holds the record plus `created_at` and `local_pdf_path` (`authors` as JSONB; `UNIQUE(source, source_id)`, unique partial index on `doi`). Each record is inserted with `ON CONFLICT DO NOTHING` and re-selected so repeated searches return stable identifiers. PDFs are validated (`%PDF-`) and stored as `raw_papers/<title>--<uuid>.pdf`; `getLibraryPapers` loads *all* rows and matches files by UUID, stored path or `source_id` substring. AI analysis is serial: each paper becomes a chat-completion request (title, abstract, task; temperature 0.3, `max_tokens` 1024) to Groq, with a minimum 2 s interval and back-off on HTTP 429; answers go to `analysis/<title>.md`.

IV. METHODOLOGY

Latency model. With T_s the settle time of adapter s, `Promise.allSettled` gives

$$T_{\text{search}} = \max_{s \in \mathcal{S}} T_s + \varepsilon_{\text{local}}. \tag{1}$$

For a per-request service time L the code implies

$$\begin{aligned} T_{\text{oa}} = T_{\text{s2}} = T_{\text{cr}} = L, \quad T_{\text{ax}} = w_{\text{ax}} + L, \\ T_{\text{epmc}} = L + \max_{i \leq k} L_i^{\text{cr}} \approx 2L, \\ T_{\text{pm}} = \max(L, \rho) + L, \quad \rho = 0.35\,\text{s}, \end{aligned} \tag{2}$$

where $w_{\text{ax}} = \max(0,\, 3\,\text{s} - \Delta)$ is the arXiv throttle wait, Δ the time since the previous arXiv call (module-global state, persisting across searches; PubMed spacing is likewise global [C]). Hence $T_{\text{search}} \approx \max(2L,\ \max(L, \rho) + L)$, with a floor of ρ at $L{=}0$, and, from the timeouts, $T_{\text{search}} \lesssim \max(3{+}30,\ 25,\ 15{+}3,\ 2{+}2{\cdot}15) \approx 33\,\text{s}$ when an adapter hangs. A search issues $N_{\text{req}} = 7 + n_{\text{epmc}}$ requests, $n_{\text{epmc}} \leq k$ being the Europe PMC hits with a DOI.

Deduplication as implemented. Keys are κ_{doi} (lower-cased DOI without resolver prefix), $\kappa_{\text{src}} = (\text{source}, \text{source_id})$, and $\kappa_h = \text{SHA256}(\text{lc}(\text{title}) \,|\, \text{lc}(\text{first author}) \,|\, \text{year})$. In input order, a record that shares a key (tested DOI, source, hash) with a *retained* record is merged into it (filling missing `pdf_url`, `abstract`, `venue`, `doi`, `citation_count`); otherwise it is retained and its keys registered. Cost is $O(n)$. *Property 1 (order dependence):* a duplicate's keys are never registered. For records a, b, c of one work with $\kappa_h(a) = \kappa_h(b) \neq \kappa_h(c)$, a DOI-less and $\kappa_{\text{doi}}(b) = \kappa_{\text{doi}}(c)$, order (a, b, c) yields two records but (b, a, c) one; the procedure is not confluent, whereas union-find over the same keys is. *Property 2 (no negative DOI evidence):* κ_h is evaluated even when both DOIs exist and differ. For a same-work pair without a shared DOI, a merge requires equal title, author and year strings; under independence, $\Pr[\text{merge}] \approx p_t p_a p_y$. We do not estimate these probabilities for real providers but measure the response to controlled values of each factor.

AI scheduling. With start interval τ (2 s) and latencies L_i, starts are separated by $\max(L_{i-1}, \tau)$, so $T_N = \sum_{i<N} \max(L_i, \tau) + L_N$. The a-th retry after HTTP 429 waits $2^a\,\text{s} + U[0, 0.5)\,\text{s}$ for $a \leq m = 3$, adding $\sum_{a=1}^{k}(2^a + 0.25)\,\text{s}$ in expectation; the $(m{+}1)$-th consecutive 429 throws.

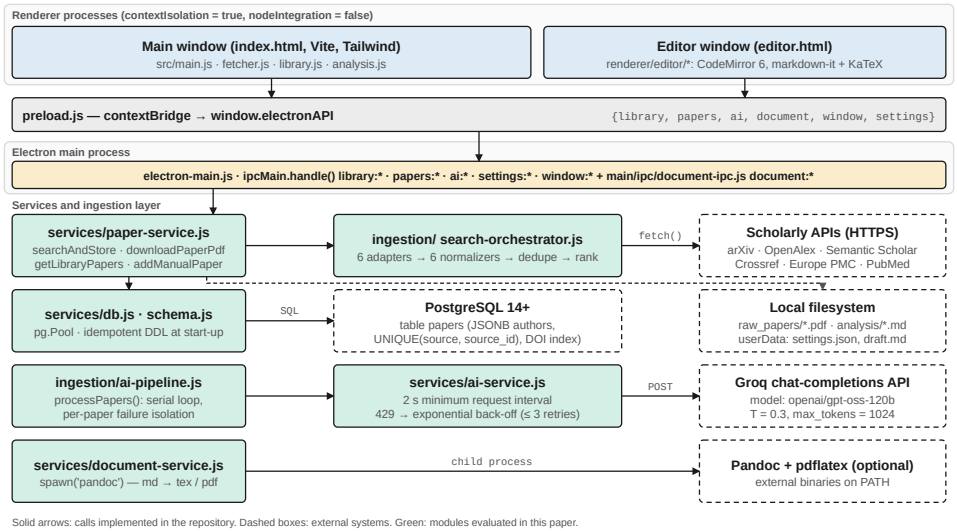


Fig. 1. **[C]** Architecture of RIE as implemented in the repository (commit `bf2d3a3`). Renderer processes reach services only through the `contextBridge` API and IPC handlers. The dashed arrow from `paper-service.js` denotes file I/O to `raw_papers/` (PDF library). Green boxes are the modules exercised by the experiments in Section VI; the editor and export path are described but not evaluated.

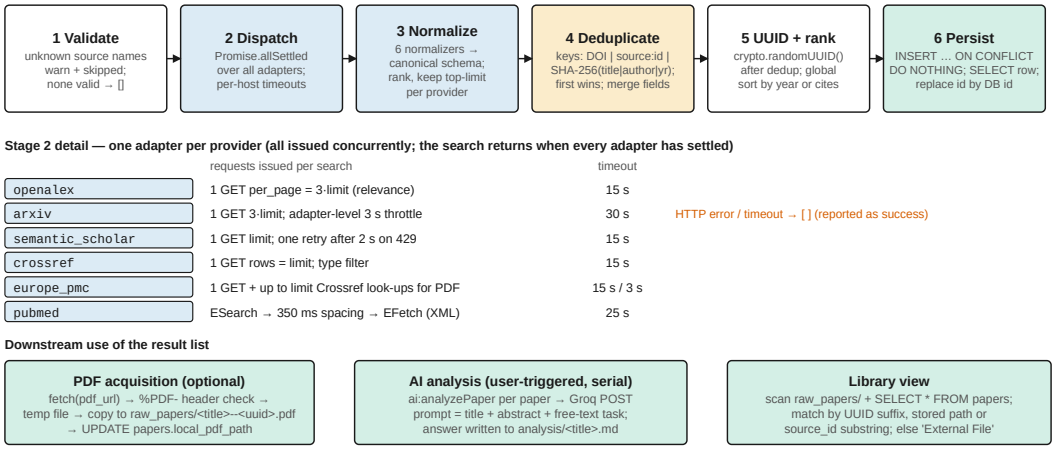


Fig. 2. **[C]** Data flow of a search (stages 1–6), the per-provider requests of stage 2, and downstream consumers. The orange annotation marks a behaviour established experimentally [S] in Section VI: an arXiv adapter failure is swallowed and appears as a successful, empty provider.

Harness. Repository modules run unmodified under Node.js; only `globalThis.fetch` is replaced by an emulator that routes by host to six provider mimics or Groq, renders a synthetic ground-truth corpus into the raw shapes the normalizers read (derived from normalizers and repository fixtures, *not* recorded traffic), and injects latency and faults (HTTP 500/429, network error, hang until abort). Database experiments use the repository schema on PostgreSQL 16.15 over loopback; randomness is seeded. Part D of E2 replicates orchestrator steps 1–3 with the same modules, and E1 times `xml2js` with the adapters' options. A verification script confirms an unchanged application, identical record sets and order between replicated and real orchestrator on three queries, and 200/200 faithful normalizer round-trips per provider (Europe PMC authors rendered "Surname Initial" by design). **Corpus and metrics.** Works have unique titles, 2–6 authors (some with diacritics), DOI, PMID, an arXiv id for half, and a preprint year $\leq$1 below the journal year. *Pairwise merge recall* is the fraction of same-work pairs collapsed, *false merges* distinct-work pairs collapsed, *residual duplicates* $|\text{output}| - |\text{works}|$. Timings use `performance.now()`; we report medians (quartiles for $n \geq 5$) after warm-up. **Platform [S].** 4 vCPU Xeon 2.3 GHz, 16 GB, Ubuntu 24.04.5, Node.js v22.23.4, PostgreSQL 16.15, `pg` 8.20.0, `xml2js` 0.6.2; no other load; Electron GUI not run. **Variants.** Besides `deduplicate()`, the harness evaluates UF-REPO (union-find over the same keys) and UF-RELAXED (union-find over DOI, `source:id`, and a diacritic-folded alphanumeric title + first-author surname and initial, linked if years differ by $\leq$1), a prototype for quantifying trade-offs. **Live session [L].** One query ("transformers"), one session, no repetition, recorded by the author from the Windows build's in-app logs. We report it as recorded, derive only arithmetic quantities, and use it solely for qualitative consistency checks.

TABLE II
EVIDENCE CLASSES USED IN THIS PAPER.

Class	Definition	Reproducible here?
[S]	Unmodified RIE modules against an emulated upstream (and real local PostgreSQL); Section VI-B to -E	Yes (`run_all.sh`)
[R]	Repository test scripts executed in the cloud sandbox; Section V	Yes, with network access
[L]	One user-recorded session of the real Windows application, from in-app logs; Section VI-F	No; not independently reproduced
[C]	Code inspection with file and line evidence; not executed	Not applicable

TABLE III
[C] SIZE OF THE CODE BASE BY LAYER (PHYSICAL LINES).

Layer	Files	Lines
Ingestion: orchestration (orchestrator, dedupe, ranking, ai-pipeline)	4	271
Ingestion: normalizers	6	720
Ingestion: source adapters	6	454
Services (db, schema, paper, ai, document)	5	746
Electron main, IPC, preload	3	431
Renderer: main-window modules	5	1 559
Renderer: editor modules	3	477
HTML views	2	1 057
Tests	4	772
Total	38	6 487

V. IMPLEMENTATION AND REPOSITORY AUDIT

RIE is a Node.js ES-module code base (Electron 40, Vite 5, Tailwind 3, CodeMirror 6, `markdown-it` with KaTeX, `pg` 8, `xml2js`); adapters use `fetch` with `AbortController` timeouts. Table III gives size per layer, and Table IV separates implemented from validated functionality.

Repository tests [R]. We executed the four test scripts twice with network access; Node's `fetch` was routed through the sandbox's HTTPS-only proxy (`NODE_USE_ENV_PROXY=1`) and a preload shim (`experiments/https_shim.mjs`) upgraded arXiv's plain-HTTP URL, with no test or application file changed. Both runs passed all 78 assertions (36/8/14/20 in `test-all-providers`, `test-ingestion`, `test-paper-service`, `test-workspace-library`). Without keys, OpenAlex and Semantic Scholar answered HTTP 429; the tests tolerate this, so a green suite does not imply every provider responded. The AI service is untested and deduplication has a single test.

Documentation drift. The README lists only arXiv and OpenAlex (six are registered), names `llama-3.3-70b-versatile` (the default is `openai/gpt-oss-120b`), lists `orchestrator.js` (actually `search-orchestrator.js`), and claims `analyses`/`collections` tables and full-text indexes that `schema.js` does not create; `src/analysis.js` still names another LLM vendor. There is no roadmap or issue list.

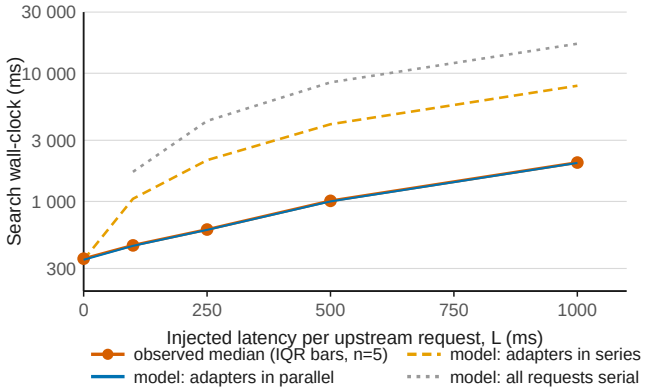


Fig. 3. **[S]** Search wall-clock vs. injected per-request latency (six providers, k=10, cold start). Observed medians (IQR bars; invisible at this scale) coincide with the parallel model; serial alternatives are shown for contrast.

VI. EXPERIMENTAL EVALUATION

All [S] results come from `experiments/run_all.sh` (raw JSON in `results/`); every quoted number is re-derived by `verification/verify_claims.mjs`. Repetitions are in Table V. The sweep was executed twice, the second time from a clean copy: counts, statuses and outputs were identical, delay-dominated timings agreed within about 15 ms, and two of 75 compared quantities (both sub-10 ms CPU microtimings) fell outside tolerance.

A. RQ1: latency, throttles and failure containment [S]

Latency structure. For a six-provider search (k=10, after $\geq$3.3 s idle) with a uniform injected latency L, the observed median matches the parallel model of Equations (1) and (2) within 4–11 ms (e.g., 2 011 vs. 2 000 ms at L=1 s), far below the sum-of-adapters and serial alternatives (8 000 and 17 000 ms; Figure 3). At L=0 a search still takes $\approx$358 ms because of PubMed's 350 ms spacing. IQRs were $\leq$1 ms.

Back-to-back searches. With L=100 ms, a second search started immediately after the first took 2 652 ms (2 642–2 656) instead of 455 ms; after gaps of 1, 2 and 3.5 s it took 1 650, 647 and 454 ms, as $w_{ax} = \max(0, 3 - \Delta)$ predicts.

Request amplification. $N_{req} = 7 + k$ (Table VI), since every emulated Europe PMC hit has a DOI and triggers an unbounded concurrent Crossref look-up; at k=50 one search emits 57 requests, 50 of them to a host that is also a selected provider.

Faults. Injecting faults into one provider at a time (Table VII), an HTTP 500 or network error is reported correctly for five providers and the others' results are unaffected. The arXiv adapter catches every error and returns `[]`, so the orchestrator reports `success, raw: 0` and cannot distinguish an outage from no hits; the repository's own `test-paper-service.js` log shows the same masking ("`[source=arxiv] failed: fetch failed`" followed by an empty success). A hung provider stalls everything until its adapter timeout (15.0, 25.0, 30.0 s for OpenAlex, PubMed, arXiv; n=3, spread <3 ms), as there is no overall deadline.

TABLE IV
IMPLEMENTATION STATUS AND SUPPORTING EVIDENCE ([R] REPOSITORY TESTS, [S] EMULATED EXPERIMENTS, [L] LIVE SESSION, [C] CODE INSPECTION).

Capability	Implemented	Evidence	Remark
Concurrent retrieval from 6 providers	yes	[R], [S], [L]	Live: four providers returned results, Semantic Scholar HTTP 429, latency not recorded; online tests passed with OpenAlex/S2 rate limited
Normalization to canonical schema	yes	[R], [S]	One hand-written fixture per provider; no recorded live payloads
Deduplication (3 keys)	yes	[R], [S]	Order dependent; fails under single-factor disagreement; one repository test
Ranking (year / citations)	yes	[S], [C]	Provider relevance order discarded; quality not evaluated
PostgreSQL persistence, stable IDs	yes	[R], [S], [L]	Live: initialized, papers shown in Library; sequential INSERT+SELECT per record
PDF download, library indexing	yes	[R], [L]	20/20 repository assertions; live retrieval worked for OpenAlex and arXiv
AI analysis (Groq, serial, retry)	yes	[S], [L]	Live: 8 papers analyzed and saved in 27 s; accuracy, cost and real rate limits not assessed
"Structured synthesis", "key contribution extraction" (README)	partly	[C]	Only a free-text prompt; no output schema
Markdown/LaTeX editor, Pandoc export	yes	[L] (start-up)	Opened in ≈1–2 s; editing and export not evaluated
`analyses`, `collections` tables; full-text indexes (README)	no	[C]	Schema creates only `papers`

TABLE V
EXPERIMENT INVENTORY (ALL CLASS [S] UNLESS NOTED). "REPS" ARE TIMED REPETITIONS AFTER WARM-UP.

ID	Measures	Sample / repetitions
E1	normalizer and XML cost	2 000 records × 9 passes; XML 20 reps
E2	deduplication (A–D)	161–500 works per probe; 1 000 works × 3 records × 6 orders; 20 queries × 2 limits
E2b	dedup scaling	$n \leq 10^5$; 7 reps (10^5: 3)
E3	orchestrator: sweep, warm start, amplification, faults, hang, CPU	5 latencies × 5 reps; 4 gaps × 5; 1 run per k; 13 fault cases; 3 hangs × 3; 5 limits × 15
E4	PostgreSQL layer	4 limits × 6; one 8-query session; 12 library sizes × 5
E5	AI client (emulated endpoint)	4 latencies × 1 run of 6 papers; 4 back-off cases × 3; 4 error codes; prompt cases
L	live session [L]	1 query, 1 session, no repetition

TABLE VI
[S] UPSTREAM HTTP REQUESTS PER SIX-PROVIDER SEARCH.

Limit k	5	10	20	50
Total requests	12	17	27	57
of which Crossref look-ups for Europe PMC	5	10	20	50

B. RQ2: deduplication [S]

Single-factor disagreement. For up to 500 works we built two same-work records, an OpenAlex-style one with DOI and a DOI-less arXiv-style one (via the real arXiv normalizer), differing in one factor (Figure 4). With a shared DOI every variant merges all pairs (recall 1.00, not plotted). Without one, `deduplicate()` and UF-REPO merge only identical and case-changed records: a one-year shift, any first-author format change ("A. Surname", "Surname A"), a trailing period, a changed subtitle separator, a dropped subtitle, diacritic folding and reversed author order each give recall 0.00. The relaxed prototype restores 1.00 except for reversed authors and dropped subtitles.

Order dependence. For the Property-1 scenario over 1 000 works (Table VIII), `deduplicate()` outputs 1 000 records for two input orders and 2 000 for four, i.e., every work is output twice depending only on provider order; both union-find variants output 1 000 for every order.

False-merge probes. (i) 500 distinct-work pairs with different DOIs but identical title, first author and year were merged by all variants (Property 2). (ii) 100 records with empty title

TABLE VII
[S] FAULT INJECTION INTO ONE PROVIDER (OTHER FIVE HEALTHY). "REPORTED" IS THE STATUS IN `PROVIDERSTATS`; "LISTED" = APPEARS IN `PROVIDERFAILURES`. THE 429 CASE INCLUDES ONE 2 S RETRY.

Fault	Provider	Reported	Listed	Time
HTTP 500 / net. error	OpenAlex, S2, Crossref, EPMC, PubMed	failed	yes	0.11–0.41 s
HTTP 500 / net. error	arXiv	success, 0 items	**no**	0.41 s
HTTP 429	Semantic Scholar	failed	yes	2.11 s
Hang	OpenAlex	failed	yes	15.02 s
Hang	PubMed	failed	yes	25.03 s
Hang	arXiv	success, 0 items	**no**	30.03 s

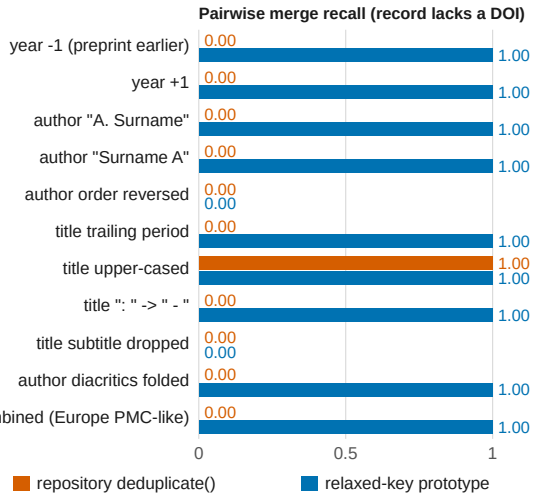


Fig. 4. **[S]** Pairwise merge recall for a same-work pair where one record lacks a DOI, by metadata perturbation (n=161–500 works each; synthetic corpus). Case-only changes are handled by the repository's lower-casing.

TABLE VIII
[S] OUTPUT RECORDS FOR 1 000 WORKS × 3 RECORDS UNDER EACH INPUT ORDER (TRUTH: 1 000). A: NO DOI; B: DOI, STRINGS IDENTICAL TO A; C: DOI, DIFFERENT STRINGS.

Input order	ABC	ACB	BAC	BCA	CAB	CBA
`deduplicate()`	2000	2000	1000	1000	2000	2000
UF-REPO	1000	1000	1000	1000	1000	1000
UF-RELAXED	1000	1000	1000	1000	1000	1000

TABLE IX
[S] MIXED END-TO-END SCENARIO (20 QUERIES, EMULATED PROVIDERS, SYNTHETIC CORPUS). RESIDUAL = OUTPUT RECORDS MINUS DISTINCT WORKS.

	k=10		k=25	
Variant	Output	Residual	Output	Residual
Raw candidates (distinct works)	1 200 (1 121)		3 000 (2 522)	
`deduplicate()`	1 130	9	2 588	66
UF-REPO	1 130	9	2 584	62
UF-RELAXED	1 121	0	2 522	0

and authors, distinct DOIs and a common year collapsed to one in every variant. (iii) 500 pairs with the same title, surname and initial but different given names ("Wei"/"Wen" Zhang) were never merged by the repository (0/500) but always by the relaxed prototype (500/500), the cost of relaxation.

End-to-end mixed scenario. Real adapters and normalizers ran through the emulator for 20 queries over 400 works (provider membership 0.5, abbreviated Europe PMC authors, 40 % preprint-year skew; Table IX). The replicated pipeline matched the real orchestrator (58 and 133 records on the checked queries). `deduplicate()` left 0.8 % (9 of 1 121) and 2.6 % (66 of 2 522) of works duplicated at k=10 and 25, all involving an arXiv record; UF-REPO removed none of the 9 and 4 of the 66; the relaxed prototype removed all with no work lost. Rates are illustrative.

C. RQ3: local cost and scaling [S]

Normalization and parsing. Normalizers cost 3.1–6.6 µs per record; parsing 100-entry arXiv (79 kB) and PubMed (122 kB) responses took 2.4–3.8 and 5.7–7.4 ms (ranges over two executions; providers varied up to 45 % between runs, e.g., OpenAlex 6.6 vs. 3.6 µs, so we make no ranking claim). Orchestrating four JSON providers took about 5 ms at k=10, 19 ms at k=100 and 32–43 ms at k=200 (n=15), linear in the 40–800 candidates and negligible next to 15–30 s timeouts.

Persistence. `searchAndStore` adds 0.5–0.7 ms per paper for the sequential INSERT+SELECT on local PostgreSQL (Table X): 28 ms for 40 papers, 188 ms for 355. Repeating a search costs about half, returns identical identifiers for all four limits and leaves the row count unchanged.

Persisted duplicates. After eight overlapping six-provider queries (k=20, 300 works), the table held 310 rows for 271 works: 39 rows (12.6 %) were duplicates. The DOI index collapses DOI-sharing duplicates, so the rest involve the 45 DOI-less rows, all arXiv (47 arXiv rows, two having gained a DOI in the merge): the RQ2 weakness carries into the permanent library.

TABLE X
[S] `searchAndStore` COST SPLIT (FOUR JSON PROVIDERS, LOCAL POSTGRESQL; MEDIANS, n=6).

k	Papers	Orchestrator	Total (cold)	Repeat	DB ms/paper
10	40	5.1	33.4	17.7	0.71
25	97	7.5	67.5	31.9	0.62
50	191	10.6	118.3	60.3	0.56
100	355	19.6	207.2	109.9	0.53

Dedup scaling and library scan. `deduplicate()` scales roughly linearly (Figure 5a): 2.3 ms for 1 000 records, 28 ms for 10 000, 626 ms for 100 000; union-find costs the same order. `getLibraryPapers()` (Figure 5b) loads the full table and, per unmatched PDF, scans all rows (about 47–52 ns per row-file pair): 4 ms (1 000 rows, no files), 51 ms (1 000 rows, 1 000 files), 1.10 s (20 000 rows, 1 000 files); loading 20 000 rows alone takes 48 ms.

D. RQ4: AI-analysis client [S]

Scheduling. With τ=2 s, six papers took 10.2, 11.0, 12.0 and 18.0 s for model latencies of 0.2, 1, 2 and 3 s, exactly as $5\max(L,\tau)+L$ predicts (Figure 6a); median start-to-start gaps were 2 002, 2 001, 2 003 and 3 004 ms. Throughput is bounded by $\max(L,\tau)$ per paper.

Retries. Success after one, two or three consecutive 429s took 2.2–2.3, 6.2–6.7 and 14.4–14.8 s (n=3), matching $\sum(2^a+0.25)$ s; four 429s produced four requests in 14.7–14.9 s and an error (Figure 6b). HTTP 400, 401, 500 and 503 were not retried, and `Retry-After` is ignored. A failure at paper 3 of 5 was isolated (4 succeeded).

Prompt construction. The user message is `Paper Title: $title\nAbstract: $abstract\n\nTask: $prompt` without null handling: a `null` abstract (possible for arXiv, PubMed, Crossref) is sent as "`Abstract: null`", an absent one as "`Abstract: undefined`", and a 20 000-word abstract is sent whole (100 060 characters).

E. Live observations from the Windows application [L]

The author ran RIE on a Windows desktop against the real providers and Groq and recorded Table XI from the in-app logs (one query, one session, not reproduced). **Not recorded:** the per-provider limit and sort order, PubMed selection, PDF counts and sizes, search latency, per-paper AI latency, whether Semantic Scholar's retry fired, the model identifier, machine, network, and Windows/Electron versions; neither relevance nor summary accuracy was assessed. Table XII compares each observation with the emulated findings.

The run worked end to end from four live providers to Groq reports, and the rate-limited provider did not block the others.

VII. DISCUSSION

Answers. *RQ1:* latency is the maximum of adapter times, but that maximum includes adapter-internal serialization (PubMed spacing, Europe PMC enrichment, arXiv throttle) and has no deadline; failure containment is correct for five providers and misreported for arXiv. *RQ2:* only DOI-sharing records merge reliably; DOI-less records merge only on identical strings, and

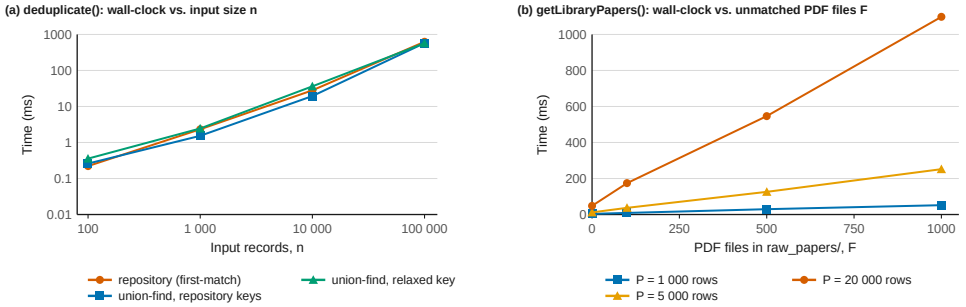


Fig. 5. **[S]** Scaling of local components. (a) `deduplicate()` and two union-find variants versus input size (log–log; medians). (b) `getLibraryPapers()` for unmatched PDF files, for three database sizes P (medians, n=5).

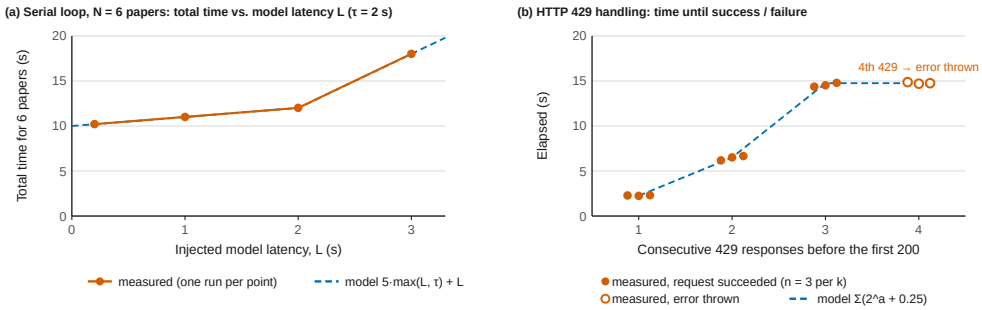


Fig. 6. **[S]** AI client scheduling against an emulated endpoint. (a) Total time for six papers vs. model latency; the first request is not delayed. (b) Elapsed time for k consecutive HTTP 429 responses followed by success; at k=4 the retry budget is exhausted and the error is thrown.

TABLE XI
USER-RECORDED LIVE OBSERVATIONS [L] (WINDOWS DESKTOP, QUERY "TRANSFORMERS"). THE MEAN PER PAPER IS DERIVED AS $27/8$; EVERYTHING ELSE IS AS RECORDED.

Item	Recorded observation
Search query	"transformers"
Providers returning results	OpenAlex, arXiv, Crossref and Europe PMC: 2 papers each, 8 papers in total
Semantic Scholar	HTTP 429 (rate limit encountered)
AI analysis	Groq pipeline analyzed all 8 papers and saved a Markdown report for each: 27 s in total
Mean time per paper	$27/8 \approx 3.4$ s (derived; covers analysis and saving)
PostgreSQL	initialized successfully; retrieved papers appeared in the Library
PDF retrieval	worked for OpenAlex and arXiv records
In-app editor	opened in approximately 1–2 s (approximate observation)

the outcome depends on input order. *RQ3:* local compute is negligible, but the library view rescans the whole table and scales with rows × files. *RQ4:* the client is τ-spaced and serial with bounded 429-only back-off, and sends `null` for missing abstracts. The live session contradicted none of these. Quantitative comparison would need per-provider and per-AI-call timestamps, which RIE does not record.

Recommendations. (1) Make the arXiv adapter reject on errors like the others. (2) Add a global deadline with partial results (hedging [19] would overload rate-limited APIs). (3) Configure a Semantic Scholar key, as the code advises. (4) Use union-find over all records' keys (same cost, Figure 5) with a DOI veto and empty-field guards. (5) Relax keys only after labelled evaluation on real data. (6) Bound the Europe PMC enrichment fan-out. (7) Guard null and over-long abstracts, honour `Retry-After`, retry transient 5xx. (8) Batch inserts and index the library scan for large libraries. (9) Call arXiv over HTTPS [C]. None was implemented or evaluated.

VIII. LIMITATIONS AND THREATS TO VALIDITY

Emulation and one live session. In [S] experiments provider latency, rate limits, errors, payload quirks and overlap are emulated; agreement with Equations (1) and (2) shows the code behaves as modelled, not how long real searches take. The only live evidence is one unreproduced session with the gaps listed in Section VI-E. **Reconstructed payloads and synthetic corpus.** Payloads are reconstructed, not recorded, so real duplicate rates may differ. Single-factor results (Figure 4) are deterministic properties of the keys, but Table IX depends on the assumed skew and author format. Unique titles make strict-key false-merge rates lower bounds. No relevance, recall or ranking quality is evaluated. **Prototypes and environment.** UF-REPO and UF-RELAXED exist only in the harness. One 4-vCPU machine and loopback PostgreSQL were used; the GUI, editor, Pandoc export, real PDF downloads (whose `fetch` has no timeout or size limit) and the security posture (plaintext keys, broad IPC surface) were only read from code.

IX. CONCLUSION

Running RIE's unmodified modules against an emulated upstream and a real PostgreSQL server characterized its latency structure (slowest adapter plus throttles, no global deadline), its misreporting of arXiv failures, its order-dependent and DOI-reliant deduplication (fixable by union-find at equal cost), its

TABLE XII
EMULATED FINDINGS [S] VERSUS THE SINGLE LIVE SESSION [L]; QUALITATIVE ONLY.

Aspect	Emulated finding [S]	Live observation [L]	Relationship
Failure containment	Injected 429 on S2 reported as failed after one 2 s retry; other providers unaffected (Table VII)	S2 returned HTTP 429; four other providers delivered 8 papers	Qualitatively consistent; retry and timing not recorded
Per-provider cap	At most k papers per provider before dedup	Four providers returned 2 papers each	Consistent with k=2 (limit not recorded); dedup effect unknown
AI scheduling	Serial; spacing $\max(L, \tau)$, τ=2 s (Figure 6a)	8 papers in 27 s (≈3.4 s each)	Consistent with a serial loop where per-paper cost exceeds τ (inference)
Persistence	INSERT/SELECT correct and fast locally (Table X)	PostgreSQL initialized; papers in Library	Functional confirmation; no timing
PDF acquisition	Not exercised in the cloud	Worked for OpenAlex and arXiv	Functional evidence only
Search latency	Parallel model, 350 ms floor, arXiv throttle (Figure 3)	Not recorded	No comparison possible
Editor start-up	Not evaluated	≈1–2 s	Approximate; no counterpart

small local costs, and its serial AI client that sends `null` for missing abstracts; a single live Windows session was consistent with these findings. Real latency distributions, overlap rates, retrieval and LLM-output quality remain open.

REPRODUCIBILITY AND AVAILABILITY

`paper_exp/` holds the sources, figures, harness (`run_all.sh` reproduces every [S] result in about 25 minutes), raw results and logs; no application file was modified (baseline `bf2d3a3`). Needs Node.js ≥18, a local PostgreSQL database `rie_bench`, and network access for [R] runs.

REFERENCES

[1] J. Priem, H. Piwowar, and R. Orr, "OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts," 2022, arXiv:2205.01833. doi: 10.48550/arXiv.2205.01833.

[2] K. Lo, L. L. Wang, M. Neumann, R. Kinney, and D. Weld, "S2ORC: The Semantic Scholar open research corpus," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4969–4983, doi: 10.18653/v1/2020.acl-main.447.

[3] R. Kinney *et al.*, "The Semantic Scholar open data platform," 2023, arXiv:2301.10140. doi: 10.48550/arXiv.2301.10140.

[4] G. Hendricks, D. Tkaczyk, J. Lin, and P. Feeney, "Crossref: The sustainable source of community-owned scholarly metadata," *Quantitative Science Studies*, vol. 1, no. 1, pp. 414–427, 2020, doi: 10.1162/qss_a_00022.

[5] C. Ferguson *et al.*, "Europe PMC in 2020," *Nucleic Acids Research*, vol. 49, no. D1, pp. D1507–D1514, 2021, doi: 10.1093/nar/gkaa994.

[6] E. Sayers, "A general introduction to the E-utilities," Entrez Programming Utilities Help, National Center for Biotechnology Information (NCBI Bookshelf NBK25497), 2022, created 2009, updated 17 Nov. 2022. Available: https://www.ncbi.nlm.nih.gov/books/NBK25497/.

[7] arXiv, "Terms of use for arXiv APIs," https://info.arxiv.org/help/api/tou.html, legacy APIs (including the arXiv API): no more than one request every three seconds, single connection. Accessed Nov. 2026.

[8] M. Visser, N. J. van Eck, and L. Waltman, "Large-scale comparison of bibliographic data sources: Scopus, Web of Science, Dimensions, Crossref, and Microsoft Academic," *Quantitative Science Studies*, vol. 2, no. 1, pp. 20–41, 2021, doi: 10.1162/qss_a_00112.

[9] M. Gusenbauer and N. R. Haddaway, "Which academic search systems are suitable for systematic reviews or meta-analyses? Evaluating retrieval qualities of Google Scholar, PubMed, and 26 other resources," *Research Synthesis Methods*, vol. 11, no. 2, pp. 181–217, 2020, doi: 10.1002/jrsm.1378.

[10] J. P. Alperin, J. Portenoy, K. Demes, V. Larivière, and S. Haustein, "An analysis of the suitability of OpenAlex for bibliometric analyses," 2024, preprint. arXiv:2404.17663. doi: 10.48550/arXiv.2404.17663.

[11] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 248:1–248:38, 2023, doi: 10.1145/3571730.

[12] X. Tang, X. Duan, and Z. Cai, "Large language models for automated literature review: An evaluation of reference generation, abstract writing, and review composition," in *Proc. 2025 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 1602–1617, doi: 10.18653/v1/2025.emnlp-main.83.

[13] I. P. Fellegi and A. B. Sunter, "A theory for record linkage," *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969, doi: 10.2307/2286061.

[14] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 19, no. 1, pp. 1–16, 2007, doi: 10.1109/TKDE.2007.250581.

[15] P. Christen, *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Berlin, Heidelberg: Springer, 2012, doi: 10.1007/978-3-642-31164-2.

[16] H. Piwowar, J. Priem, V. Larivière, J. P. Alperin, L. Matthias, B. Norlander, A. Farley, J. West, and S. Haustein, "The state of OA: a large-scale analysis of the prevalence and impact of Open Access articles," *PeerJ*, vol. 6, p. e4375, 2018, doi: 10.7717/peerj.4375.

[17] M. Shokouhi and L. Si, "Federated search," *Foundations and Trends in Information Retrieval*, vol. 5, no. 1, pp. 1–102, 2011, doi: 10.1561/1500000010.

[18] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.

[19] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[20] W. M. Bramer, D. Giustini, G. B. de Jonge, L. Holland, and T. Bekhuis, "De-duplication of database search results for systematic reviews in EndNote," *Journal of the Medical Library Association*, vol. 104, no. 3, pp. 240–243, 2016, doi: 10.3163/1536-5050.104.3.014.

[21] J. Rathbone, M. Carter, T. Hoffmann, and P. Glasziou, "Better duplicate detection for systematic reviewers: evaluation of Systematic Review Assistant-Deduplication Module," *Systematic Reviews*, vol. 4, p. 6, 2015, doi: 10.1186/2046-4053-4-6.

[22] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, arXiv:2005.11401. https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[23] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2023, arXiv:2312.10997.

[24] J. Lála, O. O'Donoghue, A. Shtedritski, S. Cox, S. G. Rodriques, and A. D. White, "PaperQA: Retrieval-augmented generative agent for scientific research," 2023, arXiv:2312.07559.

[25] M. D. Skarlinski *et al.*, "Language agents achieve superhuman synthesis of scientific knowledge," 2024, arXiv:2409.13740.

[26] J. M. Nicholson, M. Mordaunt, P. Lopez, A. Uppala, D. Rosati, N. P. Rodrigues, P. Grabitz, and S. C. Rife, "scite: A smart citation index that displays the context of citations and classifies their intent using deep learning," *Quantitative Science Studies*, vol. 2, no. 3, pp. 882–898, 2021, doi: 10.1162/qss_a_00146.

[27] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," 2025, arXiv:2508.10925.